

CS361 Assignment 1

Tomas Salaz

10 September 2025

Problem 1

A. Binary Search

```
BinSearch(A, v)
  start = 0
  end = A.length
  mid = start + (end - start) // 2    #// Floor Division
  if start > end
    return -1
  elif A[mid] = v
    return mid
  elif A[mid] > v
    BinSearch(A, v, start, mid-1)
  else
    BinSearch(A, v, mid+1, end)
```

B. Correctness

The function *BinSearch*($A, v, start, end$) returns the index of the value v if it is contained in $A[start : end]$, otherwise the algorithm returns -1 which is correct.

C. Proof

We are going to prove the correctness of *BinSearch* by induction on the Size of the Array using Start and End.

Basis: If $start > end$ the array $A[start : end]$ does not contain the value v and -1 is returned and is correct.

Induction: When $start \leq end$, The Array, $A[start : end]$ contains at least one value and we can use the midpoint, called mid , to find the value v . There are 3 cases:

1. If $A[mid] = v$ the value is at midpoint and is returned
2. If $A[mid] > v$ we return $BinSearch(A, v, start, mid - 1)$

3. If $A[mid] < v$ we return $BinSearch(A, v, mid + 1, end)$

No matter the case, the array is getting smaller. By induction, $A[start : mid - 1]$ OR $A[mid + 1 : end]$ OR $A[mid]$ must contain the value v OR the -1 if not in the array.

Problem 2

A. Euclidean Algorithm

```
gcd (a,b)
  if b = 0
    return abs(a)    #Absolute Value
  return gcd(b, a mod b)
```

B. Correctness

The function $gcd(a, b)$ returns the largest integer larger than 0 that divides both numbers.

C. Proof

We are proving that $gcd(a, b)$ returns the GCD by induction on the absolute value of b .

Basis: $b = 0$, returns the absolute value of a since the $gcd(a, 0) = 0$ if $a > 0$, trivially.

Induction: $b > 0$. Let $r = a \bmod b$. Since $a = qb + r$ the common divisors of a and b are also common divisors of b and r so $gcd(a, b) = gcd(b, r)$. So by induction on the absolute value of b is always decreasing in the recursive call $gcd(b, r) == gcd(a, b)$

Problem 3

A. Palindromic Array

```
IsPalin(A, start, end)
  start = 0
  end = A.length
  if start >= end
    return True
  if A[start] != A[end]
```

```

        return false
    return IsPalin (A, start+1, end-1)

```

B. Correctness

The function *IsPalin*(*A*, *start*, *end*) returns True if the Array, *A*[*start*:*end*] is the same when reversed (is a Palindrome). Otherwise returns False.

C. Proof

We are proving that *IsPalin*(*A*, *start*, *end*) is correct by induction on the size of the array.

Basis: If *start* ≥ *end* the code returns True since any list of 0 or 1 is a palindrome, trivially. So this is correct.

Induction: if *start* < *end* *length* > 2. If *A*[*start*] = *A*[*end*], we recur and the next array has size *A.length* − 2 so by induction, the array is getting smaller and the algorithm is correct.

Problem 4

Max Value and Occurrences Algorithm

```

MaxAndOcc(A)
    if A.length = 0
        return (0,0)
    if A.length = 1
        return (A[1], 1)
    (m,c) = MaxAndOcc(A[start + 1:end])
    max = A[0]
    if
        if x>m:
            return(x,1)      //update new max m
        elsif x == m
            return (m,c+1)  //update count c
        else
            return (m,c)

```

B. Correctness

The function *MaxAndOcc*(*A*) returns the max values and the number of occurrences, as long as the Array has 1 or more elements, otherwise returns 0.

C. Proof

We are proving that $MaxAndOcc(A)$ is correct by induction on the length.

Basis If $A.length \leq 1$ the code returns the Trivially Correct answers since either case is true.

Induction If $A.length \geq 2$ the code calls $MaxAndOcc(A[1 : n - 1])$ making the new length $A.length - 1$, and evaluates based on 3 cases:

1. If $x > m$ the New Max and Count are updated and we return the $(x, 1)$ meaning the max occurs once in the array and is correct
 2. If $x == m$ the Max M is saved and we update the count, thus this is correct
 3. Otherwise Max and Count remain unchanged
- In all cases the tuple $(Max(A), CountMax(A))$ is returned and thus correct.